

PBL Secondary Research Report

FPGA-Based Real-Time FIR Filtering

System for Multi-Channel Biomedical Signals

Chapter 1: Introduction

1.1 Background

Biomedical and environmental monitoring systems play a critical role in modern healthcare and wearable technology. These systems continuously acquire physiological signals such as Electrocardiogram (ECG), respiration rate, and pulse rate to analyze the health condition of a patient in real time. However, these signals are often affected by various types of noise such as motion artifacts, power line interference, and muscle activity noise, which degrade the accuracy of signal interpretation.

Digital Signal Processing (DSP) techniques are widely used to enhance the quality of biomedical signals. Among these techniques, Finite Impulse Response (FIR) filters are preferred due to their inherent stability, linear phase response, and suitability for hardware implementation. However, software-based FIR filtering executed on general-purpose processors introduces limitations such as higher latency, increased power consumption, and reduced efficiency in real-time applications.

To overcome these challenges, FPGA (Field Programmable Gate Array)-based hardware implementation is increasingly used. FPGA allows parallel processing and pipelined architecture, making it highly efficient for real-time signal processing tasks. In this project, although the system is conceptually designed for multiple biomedical signals (ECG, respiration, and pulse rate), the primary focus is on ECG signal processing to demonstrate real-time FIR filtering using FPGA.

1.2 Problem Statement

Existing software-based FIR filtering systems for biomedical signals suffer from high latency and increased power consumption, making them inefficient for real-time wearable and IoT applications. Most current approaches also process physiological signals such as ECG, respiration, and pulse in a sequential manner, limiting true multi-channel real-time monitoring. This project proposes an FPGA-based FIR filtering system to enable fast, low-power, and parallel real-time processing, with ECG implemented as the primary signal for validation.

1.3 Objectives

- To design and implement an FPGA-based FIR filtering system for real-time biomedical signal processing.
- To enable simultaneous (parallel) filtering of multiple biomedical signals such as ECG, respiration, and pulse rate.
- To achieve low-latency, high-speed, and power-efficient signal filtering suitable for wearable and IoT healthcare applications.
- To focus the current implementation only on the ECG signal as a representative biomedical signal, while keeping the design scalable for multi-channel extension in future.

Chapter 2: Literature Review (Secondary Research)

2.1 Review of Existing Work

Paper 1: Hardware Implementation of FIR Filter for ECG Signal Processing: Design, Optimization, and Performance Analysis on an FPGA (Uttaphan et al., 2024)

This research paper focuses on accelerating ECG denoising using an Intel Cyclone IV FPGA to overcome the serial processing limitations of software. The researchers designed a 21-tap FIR filter using the Kaiser Window method and ECG data from Physionet. By

mapping signal flow graphs into hardware logic, they achieved processing speeds up to 1,250 times faster than MATLAB implementations. The work further introduces optimized architectures using operation scheduling to reduce logic utilization and power consumption while maintaining high accuracy. Despite achieving very high processing speed, the design may still face limitations in terms of scalability, as increasing filter order can significantly raise hardware complexity. Additionally, the reliance on operation scheduling for optimization may introduce design constraints that reduce flexibility for different ECG noise conditions or real-time adaptability.

Paper 2: FPGA Implementation of High Speed FIR Low Pass Filter for EMG Removal (Leelakrishna & Selvakumar, 2013)

This research paper presents an efficient hardware architecture specifically for removing electromyogram (EMG) noise from ECG signals. The authors proposed a symmetric branched tree architecture for a 37th-order FIR filter, which reduces computational delay and increases throughput. To minimize area and cost, they utilized coefficient quantization in Q(16,14) format. Implemented on a Xilinx Virtex-5 FPGA, the design achieved high-speed performance with very low logic occupation, utilizing only 1% of the device's slice resources. Although the symmetric branched tree architecture improves speed and reduces delay, it may become less efficient when scaled to higher-order filters due to increased structural complexity. Additionally, coefficient quantization in Q(16,14) format can introduce precision loss, which may slightly affect ECG signal reconstruction accuracy.

Paper 3: High Frequency EMG Noise Removal Using FIR Low Pass Filter Based on FPGA (Bhaskar & Uplane, 2016)

The authors developed an efficient FIR filter using Distributed Arithmetic (DA) to remove muscle contraction noise (EMG) from ECG signals. This architecture replaces traditional multipliers with Look-Up Tables (LUTs) to store pre-computed values, making it highly suitable for FPGA realization. Using the MIT-BIH database for evaluation, the study compared various windowing methods and concluded that the Kaiser window provided the best Signal-to-Noise Ratio (SNR). The implementation on a Spartan 3E FPGA demonstrated significant reductions in hardware complexity and latency compared to standard DSP processors. Although the Distributed Arithmetic (DA) approach reduces multipliers and improves hardware efficiency, it increases memory usage due to LUT storage, which can limit scalability for higher-order filters. Additionally, the design is relatively less flexible for real-time adaptation since changes in filter coefficients require complete LUT reconfiguration.

Paper 4: Adaptive Modeling and Denoising of Real ECG Signal (Mishra et al., 2022)

This study investigates the use of adaptive modeling and digital filtering to extract pure ECG signals from noisy measurements. The methodology highlights the use of

second-order FIR Notch filters to specifically target power line interference and other artifacts. The authors emphasize that while software environments like MATLAB provide convenient tools for initial design and classifies ECG classified classification, custom hardware is essential for real-time applications requiring low-latency and parallel processing. Although the use of second-order FIR notch filters is effective in removing specific interferences like power line noise, it is limited in handling multiple or varying types of ECG noise simultaneously. Additionally, the study relies more on software-based adaptive modeling, which may not fully capture the constraints and optimization challenges of real-time hardware implementation.

2.2 Comparative Analysis

Existing research highlights a clear shift from software-based serial processing to parallel hardware architectures to handle large ECG datasets. While older methods focused on basic symmetry to reduce multipliers, more recent approaches utilize Distributed Arithmetic (DA) or advanced operation scheduling to minimize power and area while maximizing throughput.

Feature	Uttraphan et al. (2024)	Leelakrishna et al. (2013)	Bhaskar & Uplane (2016)
Filter Type	FIR (Kaiser Window)	FIR (Symmetric)	DA-FIR (Various Windows)
Filter Order	20 (21 taps)	37	13 (Kaiser) to 37 (Bartlett)
Architecture	Operation Scheduling	Symmetric Branched Tree	Distributed Arithmetic (DA)
FPGA Platform	Intel Cyclone IV	Xilinx Virtex-5	Xilinx Spartan 3E
Power/Area	73.49 mW / 145 LEs	1% Slice Utilization	87 Slices (Kaiser)

Processing Speed	1,250x faster than software	High speed / Reduced delay	6.011 ns per sample
Primary Goal	Real-time monitoring	EMG Removal from ECG	High Frequency Noise Removal

2.3 Research Gap Identification

Existing research on FPGA-based FIR filter implementation for ECG denoising shows significant advancements, but several important gaps remain. Most designs tend to optimize only one aspect, such as high processing speed or low area and power consumption, without achieving a proper balance between all three. For instance, some works focus on speed improvement using advanced scheduling techniques, while others emphasize reducing hardware utilization, leading to trade-offs in overall performance. Additionally, architectures like symmetric structures and Distributed Arithmetic help reduce computational complexity, but they introduce limitations such as increased memory usage or reduced flexibility in filter design. Another limitation is that many designs are application-specific, targeting only particular types of noise like EMG or powerline interference, rather than providing a generalized solution capable of handling multiple noise sources in ECG signals. Furthermore, there is limited attention given to the seamless integration between MATLAB-based filter design and FPGA hardware implementation, especially in terms of efficient coefficient handling and real-time adaptability. Therefore, improvement is needed to develop a balanced FIR filter architecture that can achieve high speed, low power consumption, and minimal hardware usage while also being flexible and suitable for real-time ECG monitoring applications.

Chapter 3: Proposed System Design

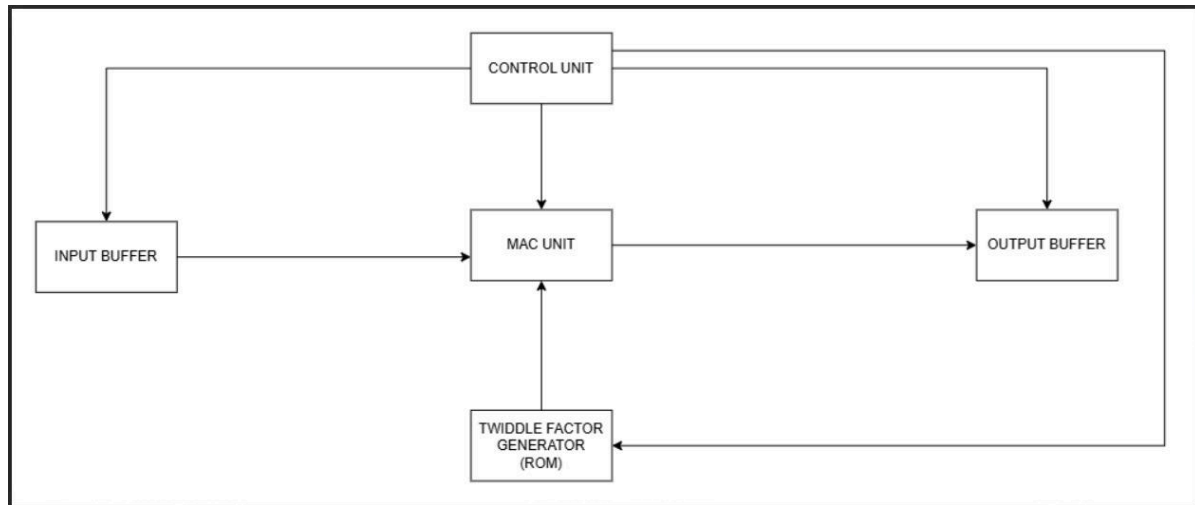
3.1 Design Justification

The proposed FIR filter architecture is selected based on a trade-off between

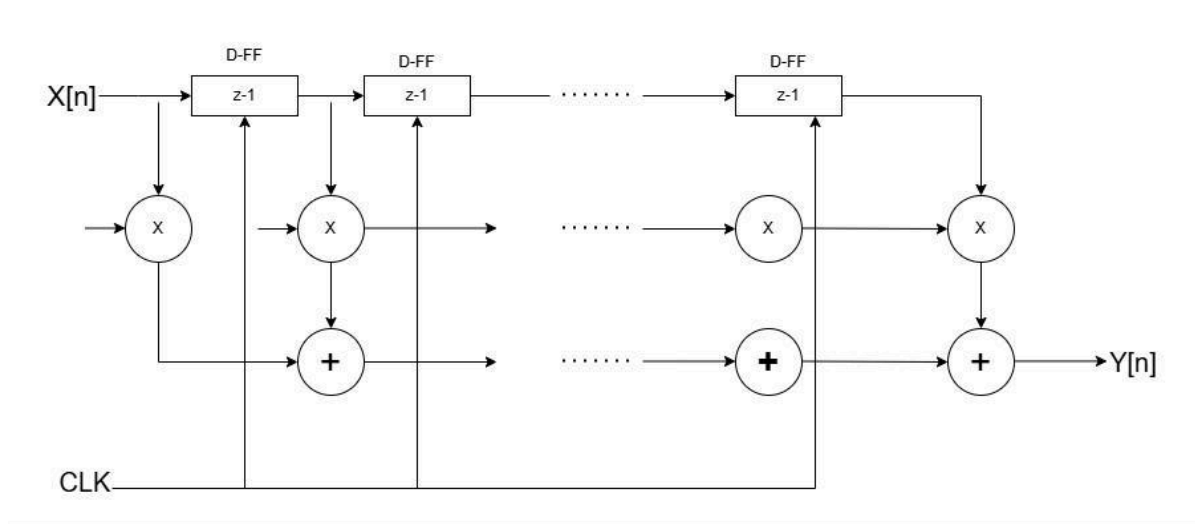
high-speed processing, hardware efficiency, and suitability for real-time ECG signal denoising. Among the available approaches in literature, architectures such as Distributed Arithmetic (DA), symmetric structures, and operation scheduling-based designs were studied. However, each has certain limitations when considered individually. DA-based designs reduce multipliers but increase memory usage due to LUT dependency, while symmetric and tree-based structures improve speed but may still consume higher hardware resources. Therefore, a balanced architecture combining **pipelined FIR structure with optimized coefficient handling and efficient adder-multiplier design** is chosen to achieve better overall performance.

This selection improves over existing literature by addressing the key trade-off issue observed in previous works. Unlike approaches that focus only on either speed or area reduction, the proposed design ensures **parallel processing with reduced critical path delay**, enabling faster computation suitable for real-time ECG monitoring. Additionally, optimized coefficient representation reduces hardware complexity compared to conventional full-precision implementations, leading to lower power consumption and improved FPGA resource utilization. Compared to earlier designs implemented on platforms like Cyclone IV, Virtex-5, and Spartan series, the proposed architecture focuses on **balanced optimization of speed, area, and power simultaneously**, making it more practical for portable and real-time biomedical applications.

3.2 Block Diagram



3.3 Architecture Diagram



Chapter 4: Leaf Cell Design

4.1 Identification of Leaf Cells

- **Shift Register (Delay Element):**

It stores previous ECG input samples to create time delay required for FIR filter operation.

- **Multiplier:**

It multiplies each delayed ECG sample with its corresponding filter coefficient (weight).

- **Adder :**

It adds all the weighted samples to produce the final filtered ECG output.

4.2 Description of Leaf Cells

1. Shift Register (Delay Element)

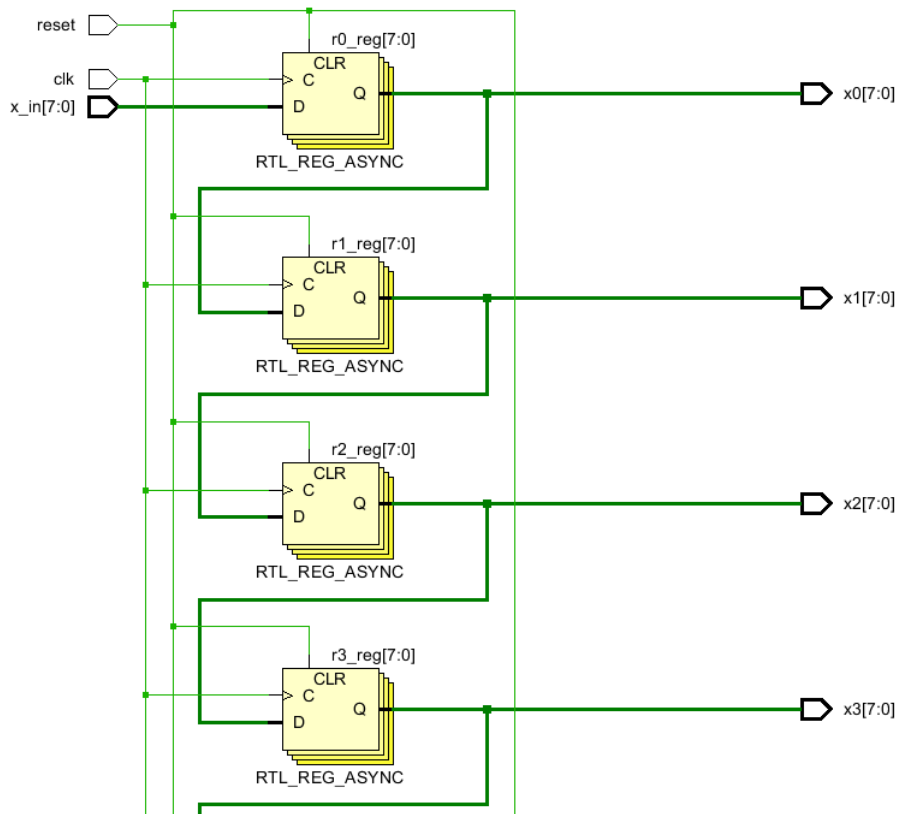
Functionality

The shift register is used to store previous ECG input samples. Since FIR

filtering depends on past and present input values, the shift register creates the required time delay by shifting data at every clock cycle.

RTL Design

The shift register is implemented using a chain of D flip-flops connected in series. Each new ECG sample enters the first register stage, while previous values are shifted forward synchronously with the clock. This forms a tapped delay line structure.



Optimization Used

- Pipeline structure for continuous data flow
- Reduced memory usage by reusing registers instead of storing full buffers
- Synchronous design to avoid timing mismatch

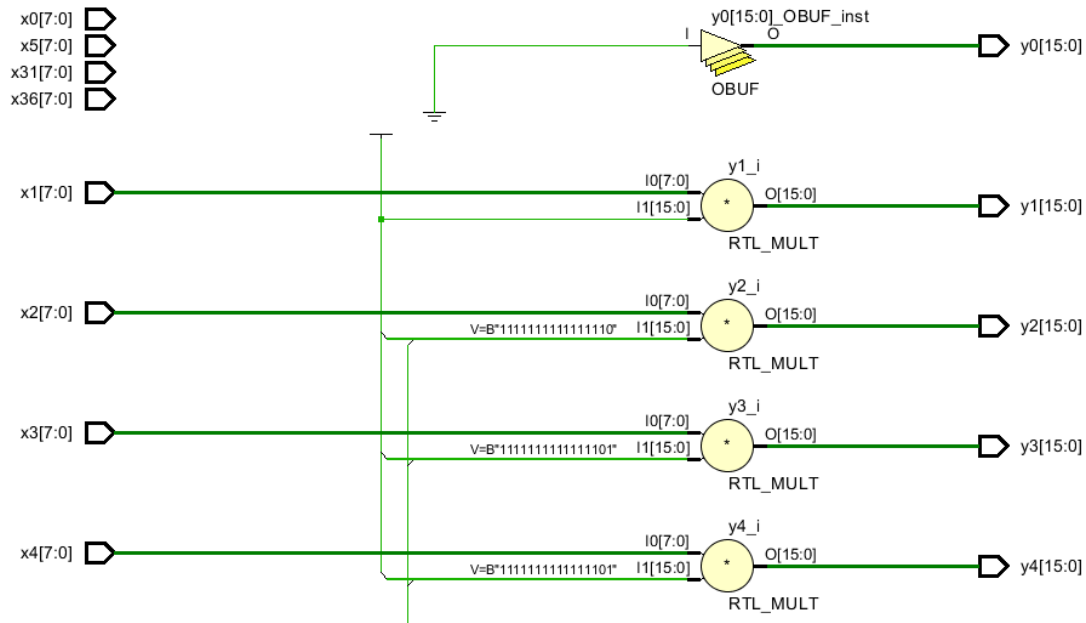
2. Multiplier

Functionality

The multiplier computes the product of each delayed ECG sample with its corresponding FIR filter coefficient. This step assigns appropriate weight to each sample for filtering noise from the ECG signal.

RTL Design

The multiplier is implemented using a combinational multiplier . Each input sample from the shift register is multiplied with a fixed coefficient stored in parameter registers.



Optimization Used

- Use of constant coefficient multiplication (can be optimized using shift-add instead of full multiplication)
- Pipelining for high-speed operation

3. Adder

Functionality

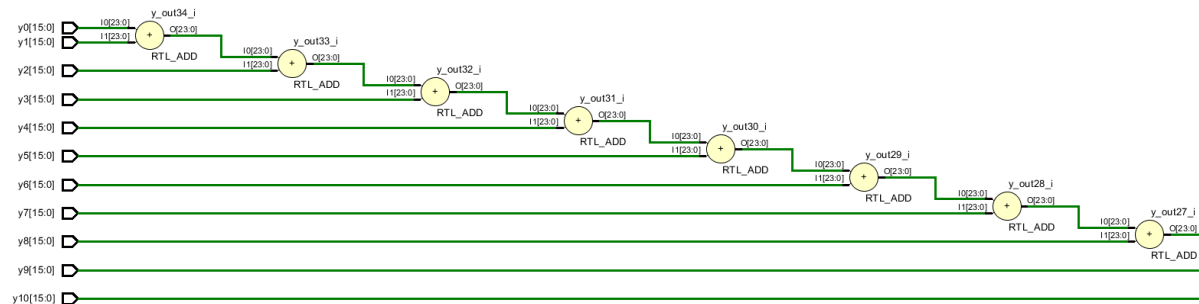
The adder sums all the multiplied outputs (weighted samples) to generate the final filtered ECG output. This represents the convolution sum in FIR filtering.

RTL Design

The adder is designed as a cascade (adder tree structure or accumulator).

Partial products from multipliers are added sequentially or in parallel

depending on architecture to produce the final output.



Optimization Used

- Adder tree structure to reduce propagation delay
- Pipelined accumulation for higher throughput
- Reduced critical path delay using balanced summation

4.3 Leaf Cell Simulation:

1.SHIFT REGISTERS(Delay Element):

MAIN CODE

```

25 module delay_line_37 (
26     input clk,
27     input reset,
28     input [7:0] x_in,
29
30     output [7:0] x0, x1, x2, x3, x4, x5, x6, x7, x8, x9,
31     output [7:0] x10, x11, x12, x13, x14, x15, x16, x17, x18, x19,
32     output [7:0] x20, x21, x22, x23, x24, x25, x26, x27, x28, x29,
33     output [7:0] x30, x31, x32, x33, x34, x35, x36
34 );
35
36 reg [7:0] r0, r1, r2, r3, r4, r5, r6, r7, r8, r9;
37 reg [7:0] r10, r11, r12, r13, r14, r15, r16, r17, r18, r19;
38 reg [7:0] r20, r21, r22, r23, r24, r25, r26, r27, r28, r29;
39 reg [7:0] r30, r31, r32, r33, r34, r35, r36;
40
41 always @(posedge clk or posedge reset) begin
42     if (reset) begin
43         r0<=0; r1<=0; r2<=0; r3<=0; r4<=0; r5<=0; r6<=0; r7<=0; r8<=0; r9<=0;
44         r10<=0; r11<=0; r12<=0; r13<=0; r14<=0; r15<=0; r16<=0; r17<=0; r18<=0; r19<=0;
45         r20<=0; r21<=0; r22<=0; r23<=0; r24<=0; r25<=0; r26<=0; r27<=0; r28<=0; r29<=0;
46         r30<=0; r31<=0; r32<=0; r33<=0; r34<=0; r35<=0; r36<=0;
47     end else begin
48         r0 <= x_in;
49         r1 <= r0;   r2 <= r1;   r3 <= r2;   r4 <= r3;   r5 <= r4;
50         r6 <= r5;   r7 <= r6;   r8 <= r7;   r9 <= r8;
51         r10 <= r9;  r11 <= r10; r12 <= r11; r13 <= r12; r14 <= r13;
52         r15 <= r14; r16 <= r15; r17 <= r16; r18 <= r17; r19 <= r18;
53         r20 <= r19; r21 <= r20; r22 <= r21; r23 <= r22; r24 <= r23;
54         r25 <= r24; r26 <= r25; r27 <= r26; r28 <= r27; r29 <= r28;
55         r30 <= r29; r31 <= r30; r32 <= r31; r33 <= r32; r34 <= r33;
56         r35 <= r34; r36 <= r35;
57     end
58 end
59

```

TEST BENCH:

```

25 module tb_delay_line_37;
26     reg clk, reset;
27     reg [7:0] x_in;
28     wire [7:0] x0,x1,x2,x3,x4,x5,x6,x7,x8,x9;
29     wire [7:0] x10,x11,x12,x13,x14,x15,x16,x17,x18,x19;
30     wire [7:0] x20,x21,x22,x23,x24,x25,x26,x27,x28,x29;
31     wire [7:0] x30,x31,x32,x33,x34,x35,x36;
32     delay_line_37 uut (
33         clk, reset, x_in,
34         x0,x1,x2,x3,x4,x5,x6,x7,x8,x9,
35         x10,x11,x12,x13,x14,x15,x16,x17,x18,x19,
36         x20,x21,x22,x23,x24,x25,x26,x27,x28,x29,
37         x30,x31,x32,x33,x34,x35,x36
38     );
39
40     always #5 clk = ~clk;
41
42     initial begin
43         clk = 0; reset = 1; x_in = 0;
44         #10 reset = 0;
45
46         // SAME SEQUENCE everywhere
47         #10 x_in = 10;
48         #10 x_in = 20;
49         #10 x_in = 30;
50         #10 x_in = 40;
51         #10 x_in = 50;
52         #10 x_in = 60;
53
54         #200 $finish;
55     end
56
57 endmodule
--

```

2.MULTIPLIER:

MAIN CODE:

```

23 module multiplier_37 (
24     input [7:0] x0, x1, x2, x3, x4, x5, x6, x7, x8, x9,
25     input [7:0] x10,x11,x12,x13,x14,x15,x16,x17,x18,x19,
26     input [7:0] x20,x21,x22,x23,x24,x25,x26,x27,x28,x29,
27     input [7:0] x30,x31,x32,x33,x34,x35,x36,
28
29     output signed [15:0] y0, y1, y2, y3, y4, y5, y6, y7, y8, y9,
30     output signed [15:0] y10,y11,y12,y13,y14,y15,y16,y17,y18,y19,
31     output signed [15:0] y20,y21,y22,y23,y24,y25,y26,y27,y28,y29,
32     output signed [15:0] y30,y31,y32,y33,y34,y35,y36
33 );
34
35 ○ assign y0 = x0 * 0;
36 ○ assign y1 = x1 * -46;
37 ○ assign y2 = x2 * -58;
38 ○ assign y3 = x3 * 60;
39 ○ assign y4 = x4 * 121;
40 ○ assign y5 = x5 * 170;
41 ○ assign y6 = x6 * 0;
42 ○ assign y7 = x7 * -314;
43 ○ assign y8 = x8 * -416;
44 ○ assign y9 = x9 * 0;
45
46 ○ assign y10 = x10 * 700;
47 ○ assign y11 = x11 * 900;
48 ○ assign y12 = x12 * 1200;
49 ○ assign y13 = x13 * 1510;
50 ○ assign y14 = x14 * 2015;
51 ○ assign y15 = x15 * 2015;
52 ○ assign y16 = x16 * 4390;
53 ○ assign y17 = x17 * 8970;
54 ○ assign y18 = x18 * 10922;
55 ○ assign y19 = x19 * 8970;
56
57 ○ assign y20 = x20 * 4390;
58 ○ assign y21 = x21 * 0;
59 assign y22 = x22 * -2015;

```

```

60 ○ assign y23 = x23 * -1510;
61 ○ assign y24 = x24 * 0;
62 ○ assign y25 = x25 * 900;
63 ○ assign y26 = x26 * 700;
64 ○ assign y27 = x27 * 0;
65 ○ assign y28 = x28 * -416;
66 ○ assign y29 = x29 * -314;
67
68 ○ assign y30 = x30 * 0;
69 ○ assign y31 = x31 * 170;
70 ○ assign y32 = x32 * 121;
71 ○ assign y33 = x33 * 60;
72 ○ assign y34 = x34 * -58;
73 ○ assign y35 = x35 * -46;
74 ○ assign y36 = x36 * 0;
75
76 ○ endmodule

```

TEST BENCH:

```

23 ○ module tb_multiplier_37;
24
25 reg [7:0] x0,x1,x2,x3,x4,x5,x6,x7,x8,x9;
26 reg [7:0] x10,x11,x12,x13,x14,x15,x16,x17,x18,x19;
27 reg [7:0] x20,x21,x22,x23,x24,x25,x26,x27,x28,x29;
28 reg [7:0] x30,x31,x32,x33,x34,x35,x36;
29
30 wire signed [15:0] y0,y1,y2,y3,y4,y5,y6,y7,y8,y9;
31 wire signed [15:0] y10,y11,y12,y13,y14,y15,y16,y17,y18,y19;
32 wire signed [15:0] y20,y21,y22,y23,y24,y25,y26,y27,y28,y29;
33 wire signed [15:0] y30,y31,y32,y33,y34,y35,y36;
34
35 multiplier_37 uut (
36     x0,x1,x2,x3,x4,x5,x6,x7,x8,x9,
37     x10,x11,x12,x13,x14,x15,x16,x17,x18,x19,
38     x20,x21,x22,x23,x24,x25,x26,x27,x28,x29,
39     x30,x31,x32,x33,x34,x35,x36,
40
41     y0,y1,y2,y3,y4,y5,y6,y7,y8,y9,
42     y10,y11,y12,y13,y14,y15,y16,y17,y18,y19,
43     y20,y21,y22,y23,y24,y25,y26,y27,y28,y29,
44     y30,y31,y32,y33,y34,y35,y36
45 );

```



```

47 initial begin
48     // SAME SEQUENCE as delay line output would produce
49     x0=60; x1=50; x2=40; x3=30; x4=20; x5=10;
50     x6=0; x7=0; x8=0; x9=0;
51
52     x10=0; x11=0; x12=0; x13=0; x14=0;
53     x15=0; x16=0; x17=0; x18=0; x19=0;
54
55     x20=0; x21=0; x22=0; x23=0; x24=0;
56     x25=0; x26=0; x27=0; x28=0; x29=0;
57
58     x30=0; x31=0; x32=0; x33=0; x34=0;
59     x35=0; x36=0;
60
61     #20 $finish;
62 end
63
64 endmodule

```

3.ADDER:

MAIN CODE:

```

22
23 module adder_37 (
24     input signed [15:0] y0, y1, y2, y3, y4, y5, y6, y7, y8, y9,
25     input signed [15:0] y10, y11, y12, y13, y14, y15, y16, y17, y18, y19,
26     input signed [15:0] y20, y21, y22, y23, y24, y25, y26, y27, y28, y29,
27     input signed [15:0] y30, y31, y32, y33, y34, y35, y36,
28
29     output signed [23:0] y_out
30 );
31
32 assign y_out =
33     y0 + y1 + y2 + y3 + y4 + y5 + y6 + y7 + y8 + y9
34     + y10 + y11 + y12 + y13 + y14 + y15 + y16 + y17 + y18 + y19
35     + y20 + y21 + y22 + y23 + y24 + y25 + y26 + y27 + y28 + y29
36     + y30 + y31 + y32 + y33 + y34 + y35 + y36;
37
38 endmodule
39

```

TEST BENCH:

```
24 module tb_adder_37;
25
26 reg signed [15:0] y0,y1,y2,y3,y4,y5,y6,y7,y8,y9;
27 reg signed [15:0] y10,y11,y12,y13,y14,y15,y16,y17,y18,y19;
28 reg signed [15:0] y20,y21,y22,y23,y24,y25,y26,y27,y28,y29;
29 reg signed [15:0] y30,y31,y32,y33,y34,y35,y36;
30
31 wire signed [23:0] y_out;
32
33 adder_37 uut (
34     y0,y1,y2,y3,y4,y5,y6,y7,y8,y9,
35     y10,y11,y12,y13,y14,y15,y16,y17,y18,y19,
36     y20,y21,y22,y23,y24,y25,y26,y27,y28,y29,
37     y30,y31,y32,y33,y34,y35,y36,
38     y_out
39 );
40
41 initial begin
42     // SAME DATA FLOW (example based on multiplier results)
43     y0=0;
44     y1=-50;
45     y2=-80;
46     y3=-90;
47     y4=-60;
48     y5=0;
49     y6=60;
50     y7=80;
51     y8=160;
52     y9=150;
53
54     // rest = 0 for simplicity
55     y10=0; y11=0; y12=0; y13=0; y14=0;
56     y15=0; y16=0; y17=0; y18=0; y19=0;
57     y20=0; y21=0; y22=0; y23=0; y24=0;
58     y25=0; y26=0; y27=0; y28=0; y29=0;
59     y30=0; y31=0; y32=0; y33=0; y34=0;
60     y35=0; y36=0;
61
62     #20 $finish;
63 end
64
65 endmodule
```

References:

Hardware Implementation of FIR Filter for ECG Signal Processing: Design, Optimization, and Performance Analysis on an FPGA

Chessda Uttraphan^{1,3,*}, Mohd Izzul Ariffi Ardani^{1,3}, Chua Wee Heng^{1,3}, Nabihah Ahmad^{1,3}, Kok Boon Ching², A Arul Edwin Raj⁴

¹ Department of Electronic Engineering, Faculty of Electrical and Electronic Engineering, Universiti Tun Hussein Onn Malaysia, 86400 Parit Raja, Batu Pahat, Johor, Malaysia

² Department of Electrical Engineering, Faculty of Electrical and Electronic Engineering, Universiti Tun Hussein Onn Malaysia, 86400 Parit Raja, Batu Pahat, Johor, Malaysia

³ VLSI and Embedded System Technology (VEST) Research Group, Malaysia

⁴ Department of ECE, Saveetha Engineering College, Chennai, India

Fpga Implementation Of High Speed Fir Low Pass Filter For Emg Removal From ECG

Leelakrishna. M¹, Selvakumar. J²

1 Research scholar, VLSI Design, SRM University, Tamilnadu, India,

2 Assistant professor, Department of ECE, SRM University, Tamilnadu, India,

High Frequency Electromyogram Noise Removal from Electrocardiogram Using FIR Low Pass Filter Based On FPGA

Mr.P.C.Bhaskar^{a*}, Dr.M.D.Uplane^b

^aDepartment of Technology, Shivaji University, Kolhapur, 416004, India.

^bDepartment of Instrumentation Science, Savitribai Phule, Pune University, Pune, 411 007, India.

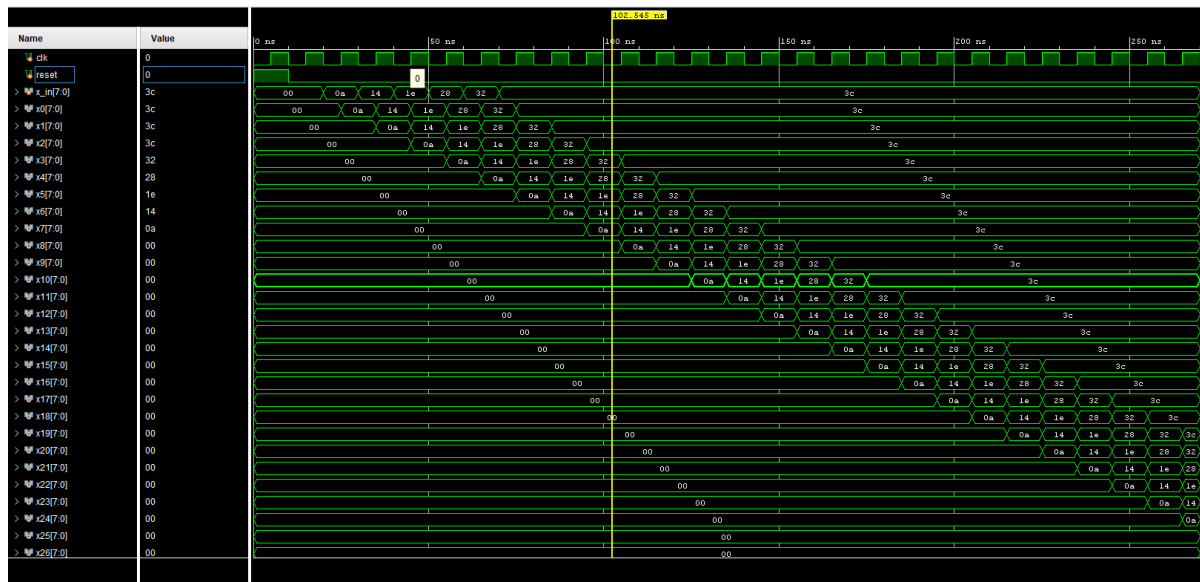
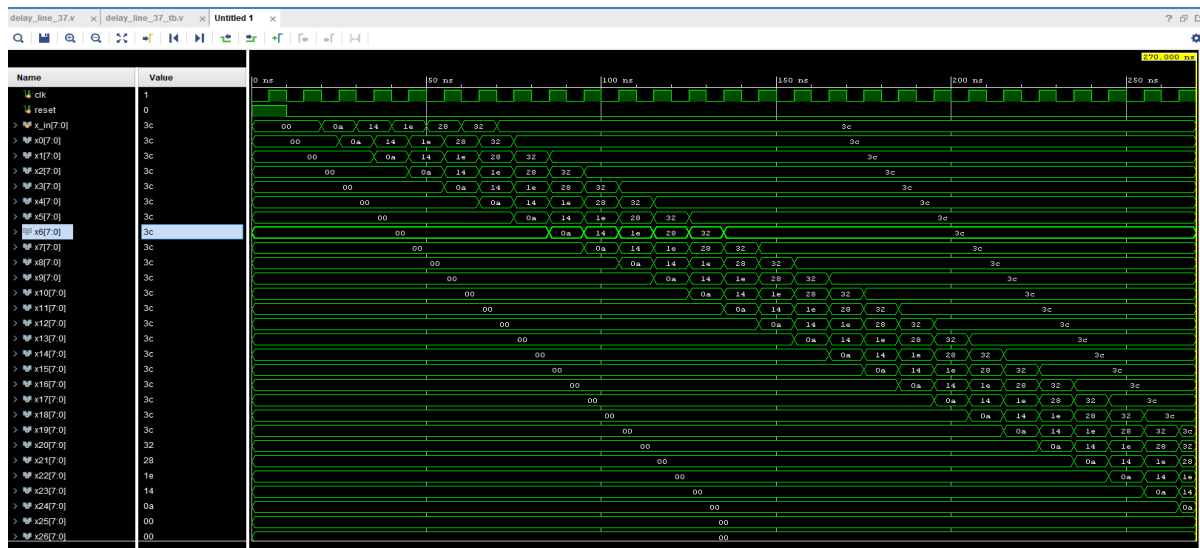
A. Mishra and S. Bhusnur, "A New Adaptive Modeling and Denoising of Real ECG signal," 2022 IEEE 3rd Global Conference for Advancement in Technology (GCAT), Bangalore, India, 2022, pp. 1-6, doi: 10.1109/GCAT55367.2022.9971940.

keywords: {Adaptation models;Finite impulse response filters;Noise reduction;Interference;Electrocardiography;Stability analysis;Splines (mathematics);ECG;Notch filter FIR;power line interference;SNR;Cubic spline;artifact},

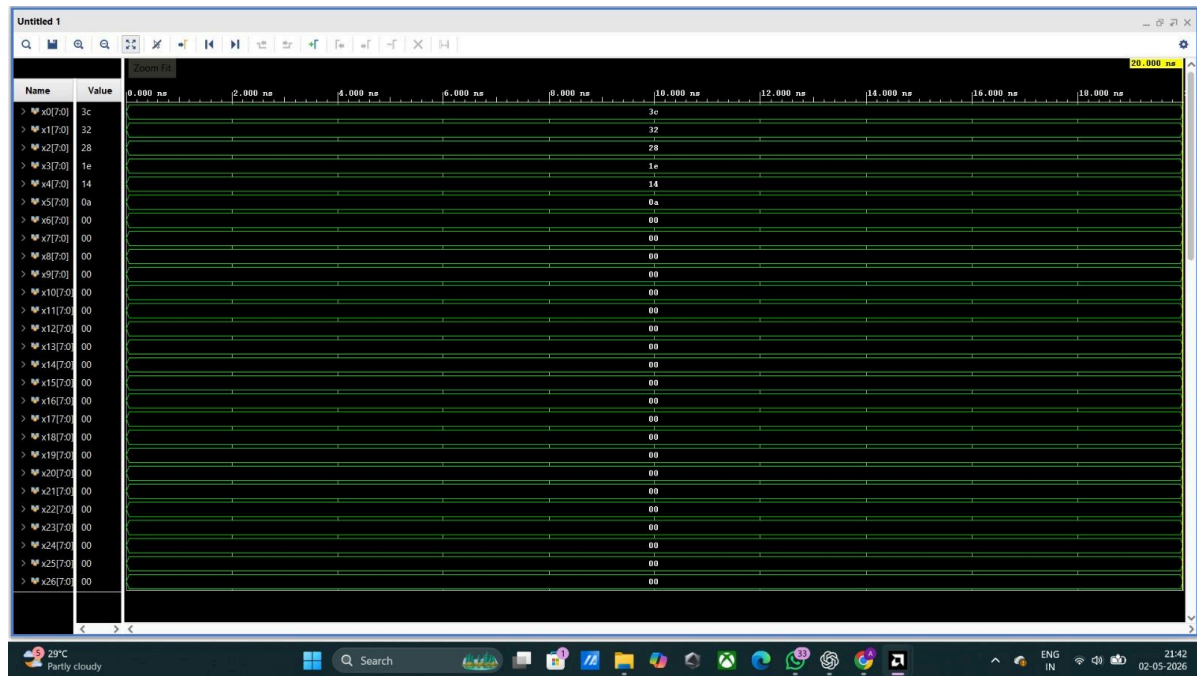
Appendix

A.1 Simulation Results

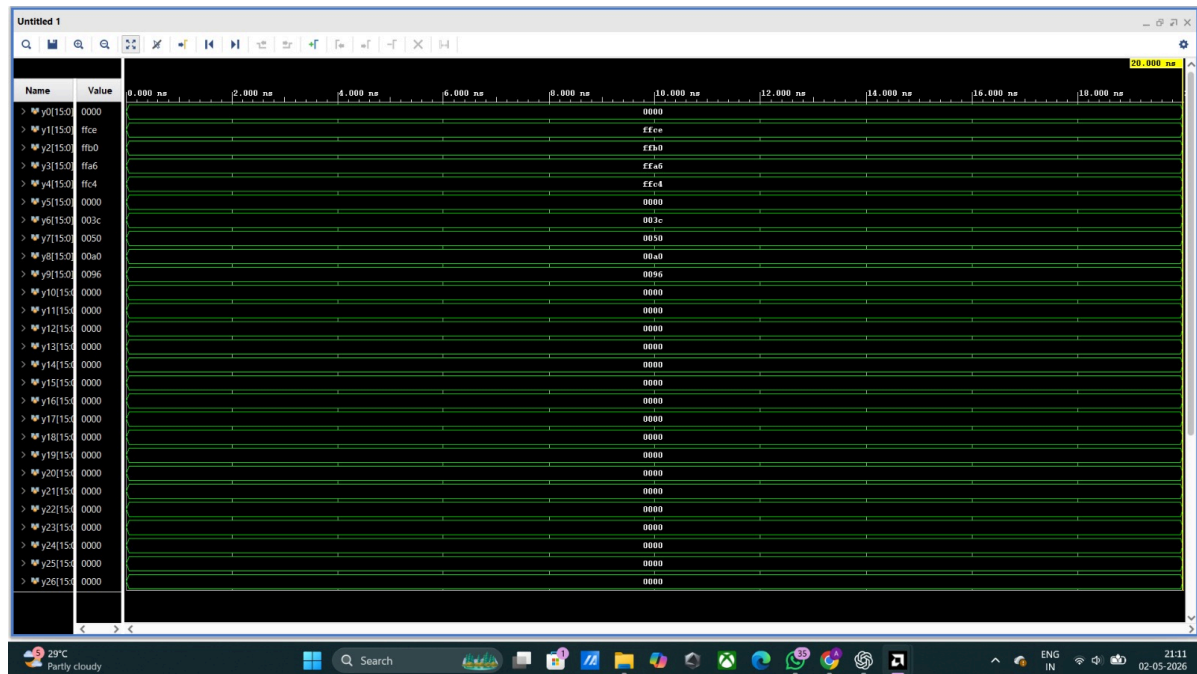
1.SHIFT REGISTERS(DELAY ELEMENT):



2.MULTIPLIER:



3.ADDER :



A.2 Tools Used

Xilinx Vivado 2018.2

MATLAB

A.3 Additional Supporting Data

Finding Digital Values using MATLAB :

```
clc;
clear;
close all;

%% Load ECG signal (example)
t = 0:1/360:10;
ecg = sin(2*pi*1*t) + 0.05*randn(size(t));

%% Step 1: Normalize (0 to 1)
ecg_min = min(ecg);
ecg_max = max(ecg);
ecg_norm = (ecg - ecg_min) / (ecg_max - ecg_min);

%% Step 2: Sampling
fs = 360;
t_sampled = t;
ecg_sampled = ecg_norm;

%% Step 3: Quantization (8-bit ADC)
n_bits = 8;
L = 2^n_bits;

ecg_digital = round(ecg_sampled * (L - 1));

%% Step 4: Reconstruction (for plotting)
ecg_reconstructed = ecg_digital / (L - 1);

%% ----- DISPLAY VALUES -----
disp('==== SAMPLE VALUES (Normalized Signal) =====');
disp('First 15 samples:');
0.4782
0.4481
0.4969
0.5250
0.5197
0.5002
0.5582
0.5610
0.5484
0.5618
0.5691
0.5788
0.5991
0.5819
0.6317

===== DIGITAL VALUES (8-bit ADC Output) =====
First 15 digital values (0 to 255):
120
114
127
134
133
128
142
143
140
143
145
148
153
148
161
```

```
C:\ECG_DATA >
2 mix x | ecg_to_verilog.m x | ECG_DATA mix x | +
ATAIECG_DATA.mlx

disp('==== SAMPLE VALUES (Normalized Signal) =====');
disp('First 15 samples:');
disp(ecg_sampled(1:15));

disp('===== DIGITAL VALUES (8-bit ADC Output) =====');
disp('First 15 digital values (0 to 255):');
disp(ecg_digital(1:15));

disp('===== TIME SAMPLES =====');
disp('First 15 time values:');
disp(t_sampled(1:15));

%% Optional: show as table (better for report)
T = table(t_sampled(1:15), ecg_sampled(1:15), ecg_digital(1:15), ...
    'VariableNames', {'Time', 'Normalized_ECG', 'Digital_ECG'});
disp('===== TABLE FORMAT =====');
disp(T);

%% ----- PLOTS -----
figure;

subplot(3,1,1)
plot(t, ecg)
title('Original ECG Signal (Analog)')
xlabel('Time (s)')
ylabel('Amplitude')

subplot(3,1,2)
plot(t_sampled, ecg_sampled)
title('Normalized Signal')

===== DIGITAL VALUES (8-bit ADC Output) =====
First 15 digital values (0 to 255):
120
114
127
134
133
128
142
143
140
143
145
148
153
148
161

===== TIME SAMPLES =====
First 15 time values:
0
0.0028
0.0056
0.0083
0.0111
0.0139
0.0167
0.0194
0.0222
0.0250
0.0278
0.0306
0.0333
0.0361
0.0389

===== TABLE FORMAT =====
```



```

disp('First 15 time values:');
disp(t_sampled(1:15));

%% Optional: show as table (better for report)
T = table(t_sampled(1:15), ecg_sampled(1:15), ecg_digital(1:15), ...
    'VariableNames', {'Time', 'Normalized_ECG', 'Digital_ECG'});
disp('===== TABLE FORMAT =====');
disp(T);

%% ----- PLOTS -----
figure;

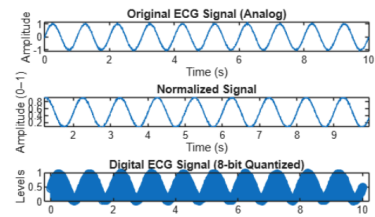
subplot(3,1,1)
plot(t, ecg)
title('Original ECG Signal (Analog)')
xlabel('Time (s)')
ylabel('Amplitude')

subplot(3,1,2)
plot(t_sampled, ecg_sampled)
title('Normalized Signal')
xlabel('Time (s)')
ylabel('Amplitude (0-1)')

subplot(3,1,3)
stem(t_sampled, ecg_reconstructed)
title('Digital ECG Signal (8-bit Quantized)')
xlabel('Time (s)')
ylabel('Levels')

```

Time	Normalized_ECG	Digital_ECG
0	0.47924	120
0.0027778	0.44811	114
0.0055556	0.49687	127
0.0083333	0.52498	134
0.011111	0.51972	133
0.013889	0.50016	128
0.016667	0.55821	142
0.019444	0.56099	143
0.022222	0.54837	140
0.025	0.56179	143
0.027778	0.56908	145
0.030556	0.57881	148
0.033333	0.59913	153
0.036111	0.58194	148
0.038889	0.63166	161



Finding Coefficient Values using MATLAB:

```

%% Given specifications
fs = 360;      % Sampling frequency
fc = 30;       % Cutoff frequency
N = 36;        % Filter order

%% Normalized cutoff frequency
Wn = fc / (fs/2); % 0.1667

%% Index
n = 0:N;
M = N/2;

%% ----- Manual sinc function -----
x = 2*Wn*(n - M);

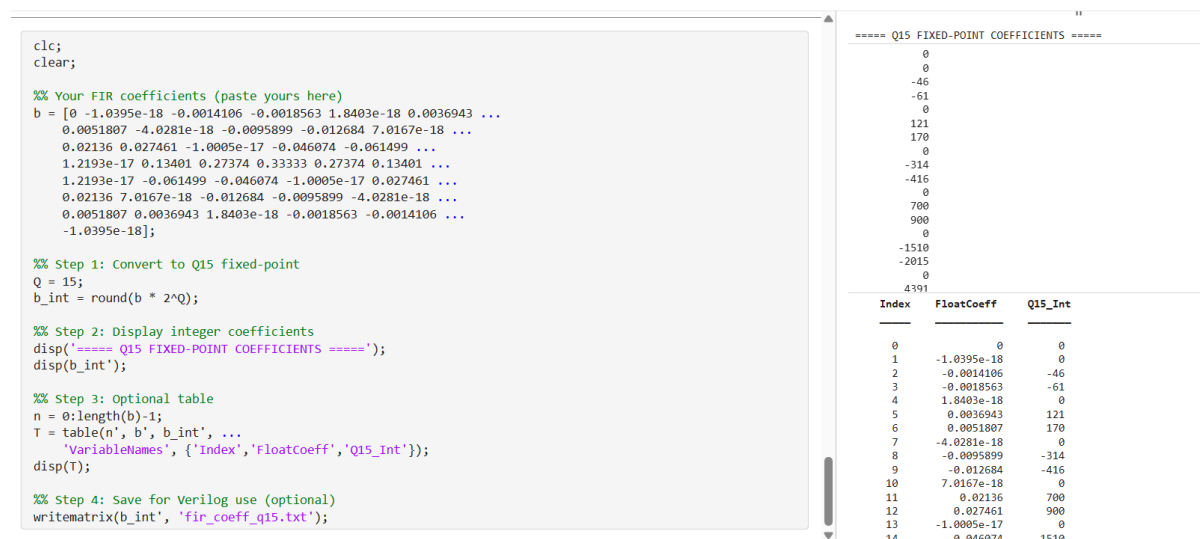
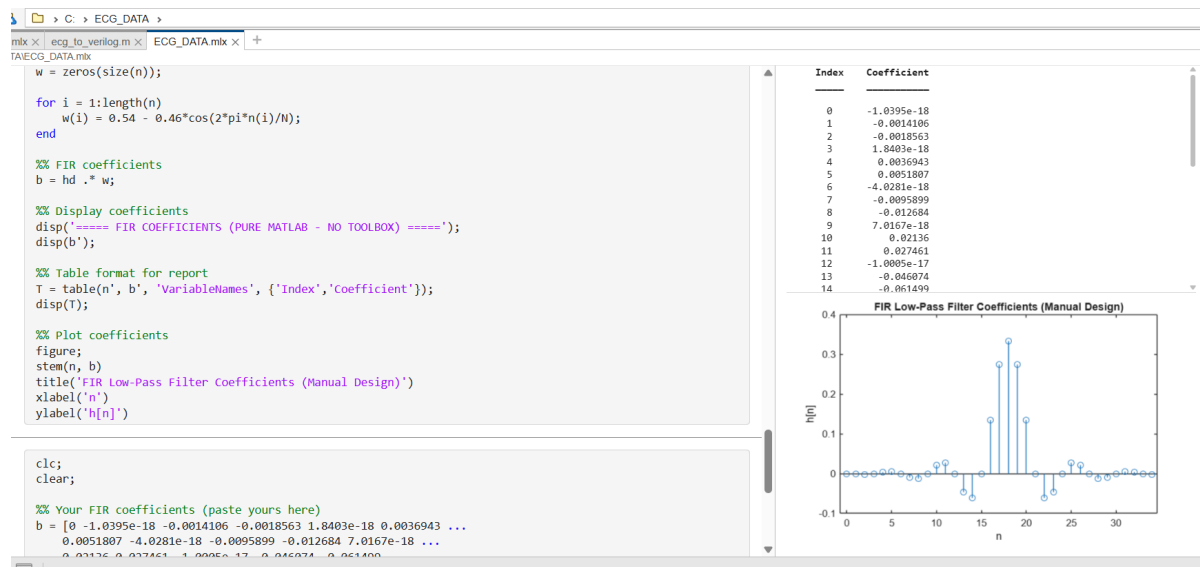
hd = zeros(size(x));

for i = 1:length(x)
    if x(i) == 0
        hd(i) = 2*Wn; % limit case
    else
        hd(i) = 2*Wn * (sin(pi*x(i)) / (pi*x(i)));
    end
end

%% ----- Manual Hamming window -----
w = zeros(size(n));

for i = 1:length(n)
    w(i) = 0.54 - 0.46*cos(2*pi*i/N);
end

```



NAME	USN
FOUZAN RASHEED	4S024EC036
FREYA JEEVITA LOBO	4S024EC037
DHRUTHWIK T	4S024EC031